Generally speaking, when a system is turned on, the CPU starts and executes the startup code from the ROM chip. After the Power-On-Self-Test (POST), the BIOS locates the boot disk, loads its bootsector (the first sector or 512 bytes) code into the RAM (location 0x7c00), and transfers the controls to it.

The program that has got the controls now is called the *first bootstrap loader* (or the first stage bootloader). This is the program that we are going to write and test in this article.

With a size constraint of 512 bytes, its only purpose would be to load the next stages in the bootloading process. But this limited space is sufficient for our experiments.

The 'Hello World' program

The Assembly code for a simple 'Hello World' program that works on the x86 architecture follows. There is no space for a detailed explanation. However, the core idea is to take each letter from a string literal (*msg*, here), and display it using the BIOS function in order to print a character.

Now just enter the following code using a text editor and save it as *hello.asm*.

```
; Set DS (data segment base) as 0x7c0
mov ax, 0x7c0
mov ds, ax

mov cx, MSGLEN
mov si, msg
mov ah, 0xe      ; BIOS 10h function code for tty output
putchar:
     mov al, [si] ; Character to be displayed
     int 0x10     ; BIOS interrupt for video service
     inc si
     loop putchar

jmp $ ; Jump here (i.e, loop forever)

msg: db 'Hello, world!'

; Let MSGLEN = Length of msg
MSGLEN: EQU ($ - msg)

; We need the boot signature as the last two bytes.
; That's why the remaining space is padded off.
padding: times (510 - ($ - $$)) db 0

BOOT_SIGN: db 0x55, 0xaa
```

One might ask: why Assembly language? There is no escaping from Assembly, at least, not in the early stages of developing an OS. Assembly is highly hardware-dependent and less productive, but it gives greater control. Also, we have a limited space of 446 bytes for our bootsector code, which is too short for a high-level program.

Getting it assembled using NASM

Although there are many assemblers available (including GNU Assembler), I prefer NASM, the Netwide Assembler, for its simplicity. Let's use the following command to assemble this code (before that, make sure you have the package *nasm* installed on your computer):

```
nasm -o hello.bin -f hello.asm
```

I am assuming the command shell is in the same directory as the source code. If not, use the command *cd* to navigate.

hello.asm is the input file, and the option *-o* specifies the output file, which is *hello.bin* (an extension has no significance, actually).

The option *-f* says that the format of the output file should be flat (plain or raw) binary. Usually, assemblers and linkers choose high-level executable formats like ELF and PE, which cannot be executed by the CPU without help from an OS. But we need something that can be directly executed by the CPU. This is why we assemble our program as *flat binary*.

Testing it with QEMU

We can use emulators and virtual machine monitors to test our bootable code without restarting the actual machine and getting out of the current OS. Let's choose QEMU as the emulator to be used in this article, for its portability. Most GNU/Linux distros provide the package *qemu*, and you can install it directly.

Now, simply run the following command (again, if the Shell is in a different directory, use *cd* to navigate first). This command instructs QEMU to start a virtual machine with the disk image *hello.bin* considered to be the boot disk.

```
qemu-system-i386 hello.bin
```

Now it works!

If you've got KVM installed, you can use the following command also:

```
kvm hello.bin
```

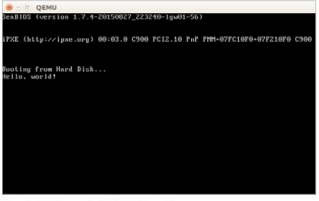

Figure 1: QEMU runs the *Hello World* bootable program